

Bubbles MVP Development Plan

This document replaces the earlier CLI-first development plan. The current Bubbles MVP is direct-API first and should not reintroduce MiniMax CLI installation, authentication, detection, or shell execution.

Product Goal

Build a macOS desktop pet assistant that lets users speak or type tasks, routes real work through direct MiniMax APIs and approved connectors, preserves memory and timeline context, and responds through one animated Bubbles avatar with chat, captions, and optional voice playback.

Current Implementation Status

- Desktop shell: Electron avatar window, assistant workspace, shared main-process state, Pixi avatar, chat, task drawer, approvals, connectors, setup, memory, timeline, and status rail.
- MiniMax setup: Token Plan key only, stored in macOS Keychain and verified through direct HTTPS calls.
- Task execution: direct MiniMax task runner emits task lifecycle events, supports cancellation, writes redacted logs, and preserves task drawer/avatar behavior.
- Voice setup: optional Gemini and OpenAI STT keys stored separately in Keychain.
- STT: renderer microphone capture plus main-process Gemini-first/OpenAI-fallback transcription.
- TTS: direct MiniMax HTTP T2A using `speech-2.8-hd`, MP3 artifacts, hex `data.audio` parsing, base64 `audio\_file` compatibility, and redacted `base\_resp` provider-error reporting.
- Agent Birth: direct MiniMax JSON generation, preview first, approval before file writes.
- Connectors: hybrid real/fixture/local connector model with visible readiness and approval policy.
- Persistence: SQLite-backed memory, timeline, approvals, and connectors with redaction on write/reopen.

Active Development Guardrails

- Keep fixture behavior stable while adding or changing real integrations.
- Preserve typed boundaries between core services, Electron IPC/preload, and renderer consumers.
- Never print raw MiniMax, Gemini, OpenAI, connector, OAuth, or task-packet secrets.
- Update tests beside changed behavior.
- For frontend or Electron-visible changes, prefer focused renderer tests and a live smoke test when practical.

Voice Plan

- Keep push-to-talk as the first interaction mode.
- Keep typed chat usable when voice is disabled or providers fail.
- Keep captions visible for spoken replies and errors.
- Keep barge-in stopping active audio playback.
- Report MiniMax TTS provider failures from `base\_resp.status\_code/status\_msg`; do not collapse them into generic no-audio text.
- Keep STT provider errors redacted and actionable.

MiniMax Plan

- Use the Token Plan key for direct MiniMax APIs.
- Default text model remains `MiniMax-M2.7`.

- Default TTS model is `speech-2.8-hd`.
- Preserve `...</think>` stripping for text/JSON responses.
- Preserve deterministic fixture artifacts for CI/demo image and music generation.

Testing Plan

Run focused checks for the touched area first:

```
```bash
npm run test:core -- setupService minimaxApiClient minimaxTaskRunner ttsService
transcriptionService voiceSetupService
npm run test:desktop -- voiceIpc SetupScreen useVoiceSession App
npm run typecheck:core
npm run typecheck:desktop
```

Run full checks before completion:

```
```bash
npm test
npm run typecheck
```

Manual QA Checklist

- Setup shows one MiniMax Token Plan form and no CLI install controls.
- Voice setup shows Gemini STT and OpenAI fallback fields.
- Microphone STT produces a user chat turn.
- Bubbles reply is captioned.
- MiniMax TTS plays an MP3 artifact when available.
- If MiniMax returns HTTP 200 without audio, the UI shows the redacted provider status reason.
- Barge-in stops active TTS playback.
- Task cancellation clears active task state and returns the avatar to neutral.
- Exported logs remain redacted.